Danmarks Tekniske Universitet

DTU

Project Report
34359 SDN Project 1: Monitoring Application

Authors

student name - Sammriddha Acharya
student number - s252707
student name - Anil Bhattarai
student number - s250300

January 4, 2026

Contents

1 Project Statement **1**

2 Project Design **2**
2.1 Project Description 2
2.2 Involved Entities and Network Relations 3
2.3 Service Behaviour Descriptions of the Monitoring Service 4
2.3.1 Operational Use Cases 4
2.3.2 Chronological Sequence of Relevant Message Exchanges 4
2.4 Definition of Expectations and Consequences 5

3 Project Implementation **5**
3.1 Implementation Environment and Tools 6
3.2 Data Plane Implementation (Mininet) 6
3.3 Control Plane Implementation (ONOS) 6
3.4 Monitoring Application Implementation 7
3.5 Link Utilization and Traffic Classification Logic 9
3.6 Data Exposure and Web Interface Integration 9
3.7 Implementation Considerations 9

4 Project Testing Strategy **10**
4.1 Test Environment 10
4.2 Functional Testing 10
4.2.1 Controller Connectivity Testing 10
4.2.2 Topology Discovery Testing 11
4.3 Traffic and Utilization Testing 11
4.3.1 Idle Network Testing 11
4.3.2 Active Traffic Testing 11
4.4 Traffic Classification Testing 11
4.5 Integration Testing 12
4.6 Robustness and Stability Testing 12
4.7 Summary of Testing Results 12

5 Results Analysis **12**
5.1 Topology and Entity Discovery Results 13
5.2 Link Utilization Measurement Results 14
5.3 Traffic Classification Results 16
5.4 Temporal Behavior and Responsiveness 17
5.5 Overall Observations 17

6 Conclusion **18**

List of Figures

1 SDN architecture diagram 2
2 Operational work-flow diagram of monitoring App 8
3 Topology shown in ONOS page 13
4 Host and Link details shown in Monitoring page 14
5 Link Utilization table in Idle condition 14
6 Link Utilization table when active (iPerf) 15
7 Traffic utilization graph in idle conditon 15
8 Traffic utilization graph when active (iPerf) 16
9 Classification of traffic according to traffic load 16
10 Monitoring App in terminal 17

1 Project Statement

Modern computer networks are required to support a wide range of applications, such as web services, real-time communication, and large data transfers, each with different performance requirements. In traditional networks, performance monitoring and measurement are complex tasks due to the distributed control logic across multiple network devices. This distribution makes it difficult to obtain a real-time and comprehensive view of network behavior and traffic flows.

Software-Defined Networking (SDN) addresses these limitations by separating the control plane from the data plane and introducing logically centralized controllers. This architecture improves network programmability, enables global visibility, and simplifies network management. SDN controllers, such as ONOS, can collect detailed information about network topology and traffic statistics. However, this information is often only accessible through low-level APIs or command-line interfaces, which are not suitable for long-term monitoring or intuitive visualization.

Currently, there is a lack of lightweight SDN-based monitoring solutions that provide a clear and continuous view of network topology and link utilization. Network operators need to understand how traffic flows through the network, identify heavily utilized links, and analyze traffic types in order to ensure efficient resource usage and maintain the desired quality of service.

To address these challenges, we propose a monitoring service that operates on top of the SDN controller and leverages its global view of the network. The service periodically collects topology and port statistics from the controller, computes link utilization, and displays the results through a web-based interface. The main objective of this work is to demonstrate how SDN can enhance network visibility through centralized, controller-driven monitoring without requiring any modifications to the underlying data-plane devices.

2 Project Design

2.1 Project Description

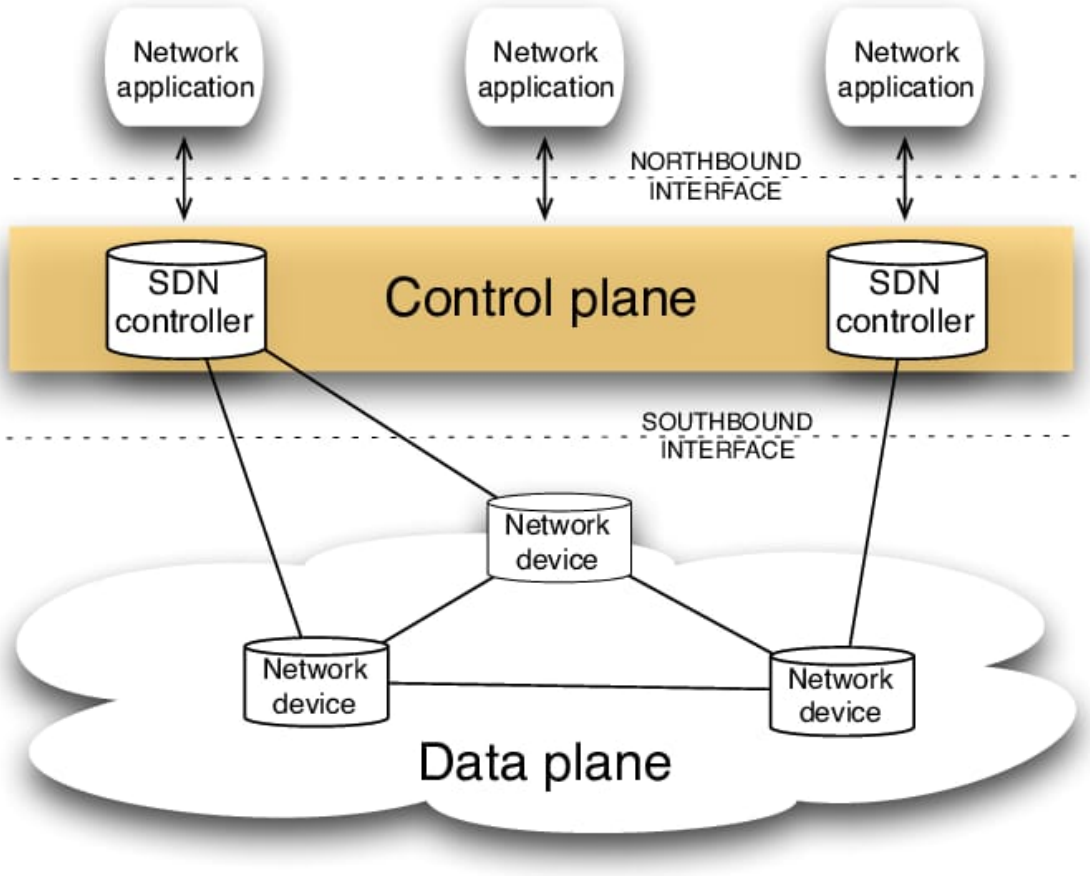

Figure 1: SDN architecture diagram

Figure 1 illustrates the Software-Defined Networking (SDN) architecture, in which the control plane is logically centralized and separated from the data plane. This separation enables network programmability and global visibility through standardized northbound and southbound interfaces [1,2].

The proposed solution is an SDN-based network monitoring service that runs on top of a centrally managed SDN controller. The service leverages key SDN architectural principles, particularly the separation of the control plane from the data plane and the controllerâs global view of the network, to provide the functionality of a general network monitoring system.

The SDN controller used in this work is ONOS, which serves as the control part of the network. It manages the network and gathers information about the network structure and how much traffic is moving through it from the devices that forward data. The data part of

the network is simulated using Mininet, a tool that creates a virtual network with switches and computers that support OpenFlow. The monitoring service is a separate program that connects to the ONOS controller through a special web-based communication method called the northbound REST interface.

One common problem in regular networks is that it's hard to see how much the network is being used. This work uses the port statistics from the ONOS controller to help with that. In traditional setups, monitoring tools collect data from each device individually. But with SDN, the monitoring service can get all the needed data directly from the controller at set times. This data is then used to figure out how much data is moving across each link, and the results are shown through a web page. This setup makes monitoring easier and shows how services can be built around the controller without changing the devices that handle the data flow.

2.2 Involved Entities and Network Relations

The system consists of many entities, each with clearly defined roles:

- **Mininet Hosts** — These represent end systems that generate and receive network traffic. Hosts are connected to the edge ports of SDN switches.
- **Mininet Switches (Open vSwitch)** — These switches operate as data-plane forwarding devices. They forward packets based on flow rules installed by the controller and maintain port-level byte counters.
- **ONOS Controller** — The ONOS controller performs a centralized role in the network infrastructure. It:
  - discovers devices, links, and hosts, and maintains a global view of the network topology;
  - collects and maintains port statistics for connected switches; and
  - provides REST APIs to expose network state to external applications.
- **SDN Service (Monitoring Application)** — This application operates above the ONOS controller. It periodically retrieves topology and statistical data from ONOS, computes link utilization metrics, and provides monitoring functionality.
- **User Interface (Web-Based)** — The user interface presents monitoring results to the user, including network topology, host locations, link utilization, and traffic classifications.

From a topological perspective, hosts are the only devices connected directly to switch devices through network links. From an operational perspective, the ONOS controller logically resides above the switches and manages their behavior. The monitoring application operates above the controller and interacts solely with the abstractions provided by ONOS, rather than communicating directly with the data-plane devices.

2.3 Service Behaviour Descriptions of the Monitoring Service

2.3.1 Operational Use Cases

The monitoring service is designed to support the following primary operational scenarios:

1. **Idle Network Scenario**
   In the absence of user-generated traffic, the monitoring service reports very low link utilization values. These values are primarily influenced by minimal control-plane traffic, such as topology discovery messages and ARP packets.

2. **Active Traffic Scenario**
   When user-generated traffic is present, for example through tools such as iPerf, the monitoring service detects increased throughput across all links along the forwarding path. As a result, higher link utilization values are computed and displayed in the user interface.

3. **Topological Change Scenario**
   When hosts or links are added to or removed from the network, the monitoring service reflects these changes during the next polling cycle. The ONOS controller maintains an updated view of the network topology, ensuring that any modifications are accurately captured and presented.

2.3.2 Chronological Sequence of Relevant Message Exchanges

The following describes the chronological sequence of the primary message exchanges between system entities:

Switch–Controller Communication (Southbound)

- Mininet switches establish OpenFlow sessions with the ONOS controller.
- ONOS periodically sends LLDP packets to discover and maintain link information.
- Switches report port-level traffic statistics back to the ONOS controller.

Controller–Application Communication (Northbound)

- The monitoring application issues HTTP GET requests to ONOS REST API endpoints.
- ONOS responds with JSON-formatted data describing devices, links, hosts, and port statistics.

Application–User Interaction

- The web-based user interface periodically requests monitoring data from the monitoring application.
- The application returns processed monitoring information in JSON format.
- The user interface updates the displayed tables and visual elements accordingly.

This sequence highlights the clear separation between southbound control interactions and northbound application interactions, which is a fundamental principle of Software-Defined Networking architectures.

2.4 Definition of Expectations and Consequences

The primary outcome of the proposed monitoring service is to provide immediate visibility into network activity through centralized SDN abstractions. Since the SDN controller maintains a global view of the network, the service enables operators to monitor all network segments from a single, centralized location, rather than relying on separate monitoring mechanisms for individual devices or network regions.

From an operational perspective, the monitoring service enables an organization to:

- identify the most highly utilized network links;
- observe the impact of traffic flows across different segments of the network; and
- gain a high-level understanding of overall traffic behavior.

From a broader SDN perspective, this project demonstrates that network monitoring can be effectively implemented as a controller-based service. By doing so, it reinforces key SDN advantages, including programmability, abstraction, centralized control, and simplified network management.

3 Project Implementation

This section describes the practical implementation of the proposed SDN-based monitoring service. It outlines the technologies used, the functional components of the system, and the realization of the monitoring logic. The implementation follows the architectural principles discussed in the Project Design section and adheres to SDN best practices by clearly separating the data plane, control plane, and application plane.

3.1 Implementation Environment and Tools

The implementation is carried out in a virtualized SDN laboratory environment to ensure reproducibility and ease of experimentation. The following tools and technologies are used:

- **Mininet** is used to emulate the data-plane network, including hosts, OpenFlow-enabled switches, and links.
- **ONOS** is deployed as the SDN controller and acts as the centralized control plane.
- **Python with Flask** is used to implement the monitoring application operating above the controller.
- **RESTful APIs with JSON encoding** are used for communication between the monitoring application and the ONOS controller.
- **HTML and JavaScript** are used to implement the web-based monitoring interface.

This setup reflects a typical SDN development workflow, where network behavior is tested in an emulated environment before deployment on physical infrastructure.

3.2 Data Plane Implementation (Mininet)

The data plane is implemented using Mininet, which emulates a network of hosts and switches interconnected by virtual links. Each switch is implemented using Open vSwitch and configured to support the OpenFlow protocol.

Mininet switches establish control connections with the ONOS controller, allowing ONOS to manage forwarding behavior and collect traffic statistics. Hosts are connected to the edge ports of switches and are used to generate traffic, for example through performance measurement tools such as iPerf. The use of Mininet enables controlled generation of traffic patterns while observing their effects across multiple network links.

3.3 Control Plane Implementation (ONOS)

ONOS operates as the centralized control plane and is responsible for maintaining a global and consistent view of the network. Upon startup, ONOS automatically discovers switches through the OpenFlow handshake process and assigns a unique identifier to each device. Link discovery is performed using periodic LLDP messages, while hosts are discovered when they begin generating traffic.The ONOS controller follows standard SDN design principles and exposes network state to external applications via northbound REST interfaces [3].

In addition to topology management, ONOS continuously collects port-level traffic statistics from connected switches. These statistics include transmitted and received byte counters for each switch port. ONOS exposes this information through its northbound REST API, allowing external applications to retrieve real-time network state without requiring direct interaction with the data plane.

3.4 Monitoring Application Implementation

The monitoring service is implemented as an external application using the Flask framework. The application operates independently of the ONOS controller and interacts with it exclusively through REST APIs, thereby preserving modularity and loose coupling. Such controller-based monitoring approaches are consistent with SDN application-layerdesign, where external services consume abstracted network state without direct access to data-plane devices [2].

A background polling mechanism is implemented using a dedicated execution thread. At fixed time intervals of five seconds, the application retrieves network devices, network links, hosts and their attachment points, as well as port-level traffic statistics for each device from the ONOS controller.

To compute traffic rates, the application stores previously collected byte counters for each device–port pair. Throughput is calculated using a delta-based method, where the difference in transmitted bytes between two consecutive polling cycles is divided by the elapsed time and converted into bits per second. Only transmitted byte counters are considered in this calculation to simplify the measurement process and avoid double-counting traffic on bidirectional links. A flow chart of how the monitoring app functions is given below.

To compute traffic rates, the monitoring application stores previously collected byte counters for each device–port pair. Throughput is calculated using a delta-based method, where the difference in transmitted bytes between two consecutive polling intervals is divided by the elapsed time and converted into bits per second.

The throughput T for a given port is calculated as:

$$T = \frac{(\Delta B \times 8)}{\Delta t} \tag{1}$$

where ΔB represents the difference in transmitted bytes between two polling intervals, and Δt denotes the time elapsed between the measurements in seconds. Only transmitted byte counters are considered in this calculation to simplify measurement and avoid double-counting traffic on bidirectional links.

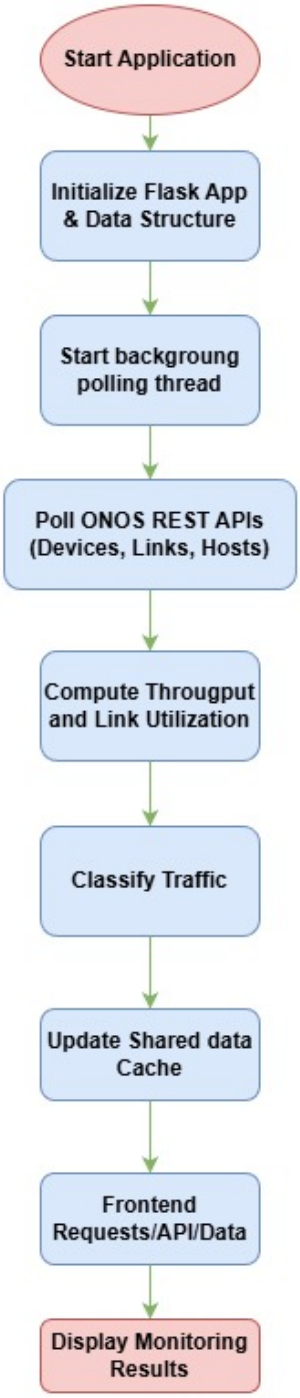

Figure 2: Operational work-flow diagram of monitoring App

Figure 1 shows the complete operational workflow of the monitoring application, showing how data is collected, processed, classified, and presented to the user.

3.5 Link Utilization and Traffic Classification Logic

Link utilization is estimated by normalizing the measured throughput with respect to an assumed link capacity. In this implementation, all links are assumed to have a fixed capacity of **1 Gbps**. Link utilization is calculated as the ratio between the measured throughput and the assumed link capacity. The resulting utilization value is capped at 100% to prevent unrealistic values caused by short sampling intervals or counter fluctuations. Link utilization is estimated by normalizing the measured throughput with respect to an assumed link capacity. In this implementation, all links are assumed to have a fixed capacity of 1 Gbps.

The link utilization U is calculated as:

$$U = \frac{T}{C} \tag{2}$$

where T is the measured throughput in bits per second and C is the assumed link capacity. The resulting utilization value is capped at 1 (100%) to prevent unrealistic values caused by short sampling intervals or counter fluctuations.

In addition to utilization metrics, the monitoring service implements a lightweight traffic classification mechanism based on predefined throughput thresholds:

- **Background Traffic**: throughput below 1 Mbps
- **Interactive Traffic**: throughput between 1 Mbps and 10 Mbps
- **Bulk Data Traffic**: throughput between 10 Mbps and 100 Mbps
- **High-Bandwidth Traffic**: throughput greater than 100 Mbps

These threshold values provide a coarse-grained but meaningful distinction between different traffic behaviors commonly observed in IP networks.

3.6 Data Exposure and Web Interface Integration

The processed monitoring data is stored in an in-memory data structure within the monitoring application and exposed through a REST endpoint that returns JSON-formatted data. This endpoint provides access to topology information, throughput measurements, utilization values, and traffic classification results.

The web-based user interface periodically queries this endpoint and refreshes its displayed content every five seconds. Network links, host information, throughput values, and utilization metrics are presented in tabular form, while time-series graphs are used to visualize utilization trends over time. This design provides near real-time visibility into network behavior without requiring direct interaction with the SDN controller.

3.7 Implementation Considerations

The implementation emphasizes simplicity, modularity, and clarity. By relying on abstractions and REST APIs provided by ONOS, the monitoring service avoids device-specific

dependencies and remains portable across different SDN environments. The use of periodic polling and fixed-interval web page refresh introduces minimal overhead while still providing timely and consistent visibility into network conditions. The implementation assumes uniform link capacities of 1 Gbps, relies solely on transmitted byte counters for utilization estimation, and uses periodic polling at five-second intervals. These assumptions simplify implementation while still demonstrating SDN-based centralized monitoring.

4 Project Testing Strategy

This section describes the testing methodology adopted to validate the correctness, robustness, and functionality of the proposed SDN-based monitoring service. The testing strategy focuses on verifying individual system components as well as evaluating the integrated behavior of the complete system within an emulated SDN environment.

4.1 Test Environment

All tests are conducted in a controlled SDN laboratory setup consisting of:

- Mininet for emulating hosts, switches, and network links;
- ONOS as the centralized SDN controller;
- a Flask-based monitoring application running externally; and
- a web browser used to access and validate the monitoring interface.

This environment enables reproducible experiments and controlled generation of traffic for evaluation purposes.

4.2 Functional Testing

Functional testing is performed to verify that each system component behaves as expected.

4.2.1 Controller Connectivity Testing

- Verified that the monitoring application can successfully authenticate and communicate with ONOS REST APIs.
- Confirmed correct retrieval of devices, links, and hosts using REST endpoints.
- Ensured that temporary controller unavailability or invalid responses do not cause the application to crash.

4.2.2 Topology Discovery Testing

- Validated that newly discovered switches and links are correctly displayed in the monitoring interface.
- Tested host discovery by generating traffic from Mininet hosts.
- Verified that topology updates are reflected after subsequent polling cycles.

4.3 Traffic and Utilization Testing

Traffic-based testing is conducted to evaluate the accuracy of throughput and utilization calculations.

4.3.1 Idle Network Testing

- Observed baseline utilization when no user-generated traffic is present.
- Confirmed the presence of minimal background traffic caused by control-plane protocols such as LLDP and ARP.
- Verified that reported utilization values remain low and stable.

4.3.2 Active Traffic Testing

- Generated traffic between hosts using iPerf.
- Verified increased utilization across all links along the forwarding path.
- Confirmed that utilization decreases once traffic generation stops.
- Ensured consistent behavior across multiple test executions.

4.4 Traffic Classification Testing

To validate the traffic classification logic, the following tests were conducted:

- Generated traffic with varying intensities using iPerf.
- Observed changes in classified traffic types based on throughput thresholds.
- Verified that classification labels update dynamically as traffic conditions change.

These tests confirm that the application correctly enriches raw traffic statistics with semantic classification information.

4.5 Integration Testing

Integration testing focuses on the interaction between all system components:

- Verified synchronized operation between the background polling thread and the web interface.
- Ensured that concurrent access to shared data structures does not introduce inconsistencies.
- Confirmed that the user interface remains responsive during continuous polling and traffic generation.

4.6 Robustness and Stability Testing

The monitoring application is evaluated for robustness and stability over extended execution periods:

- Verified that long-running polling does not result in memory leaks or performance degradation.
- Tested application behavior under temporary REST API failures or delayed responses.
- Ensured that monitoring resumes automatically after transient errors.

4.7 Summary of Testing Results

Testing confirms that the monitoring service functions correctly across a range of network conditions and traffic scenarios. The application consistently retrieves the current network state from ONOS, computes accurate utilization metrics, and presents up-to-date monitoring data through the web-based interface. Based on these results, the proposed SDN-based monitoring approach is shown to be effective and reliable.

5 Results Analysis

This section presents an analysis of the results obtained from deploying and executing the SDN-based monitoring service within the Mininet and ONOS environment. The evaluation focuses on the accuracy of network topology discovery, the precision of link utilization measurements, the behavior of traffic classification mechanisms, and the overall system responsiveness under varying network conditions.

5.1 Topology and Entity Discovery Results

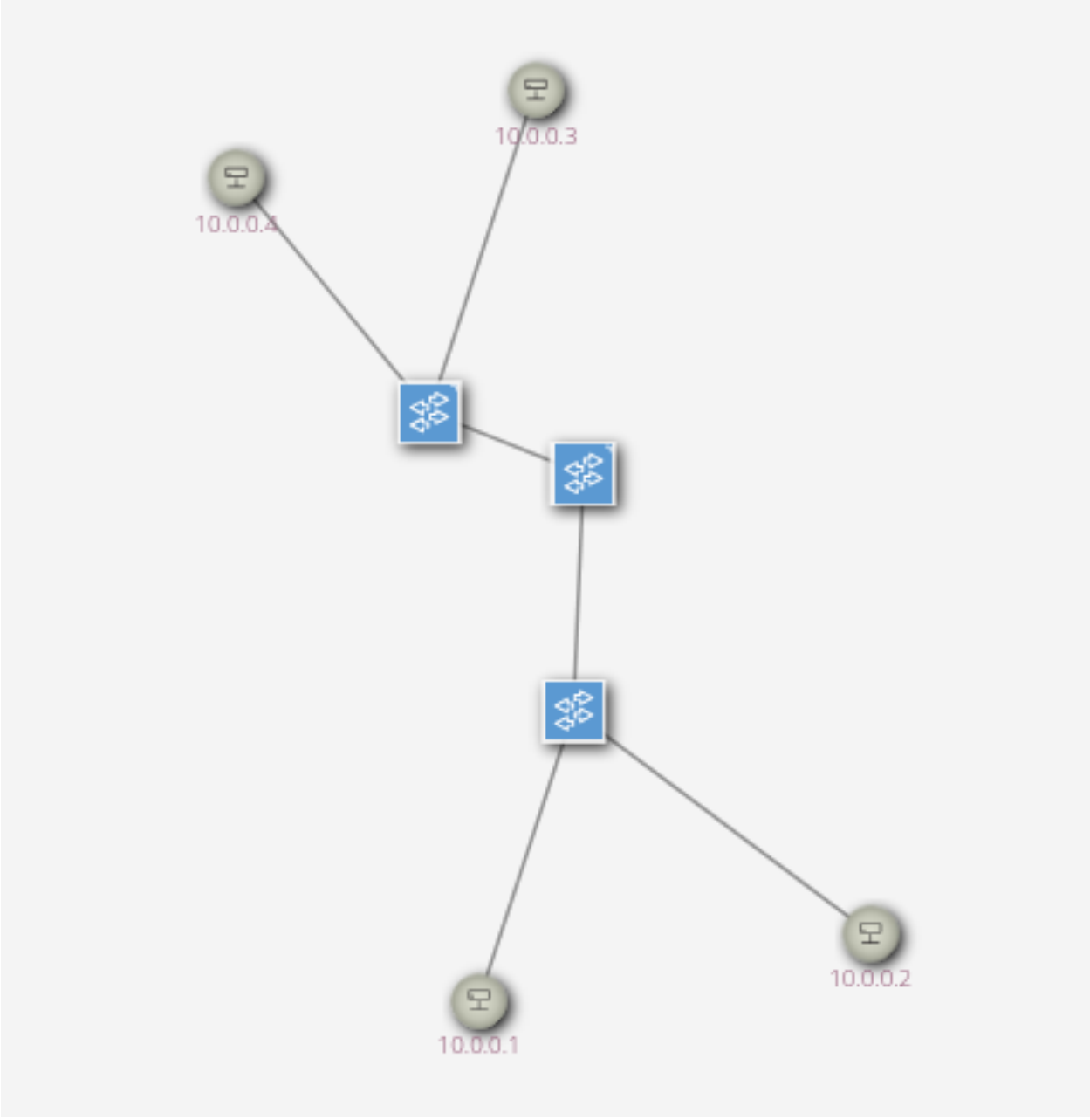

Figure 3: Topology shown in ONOS page

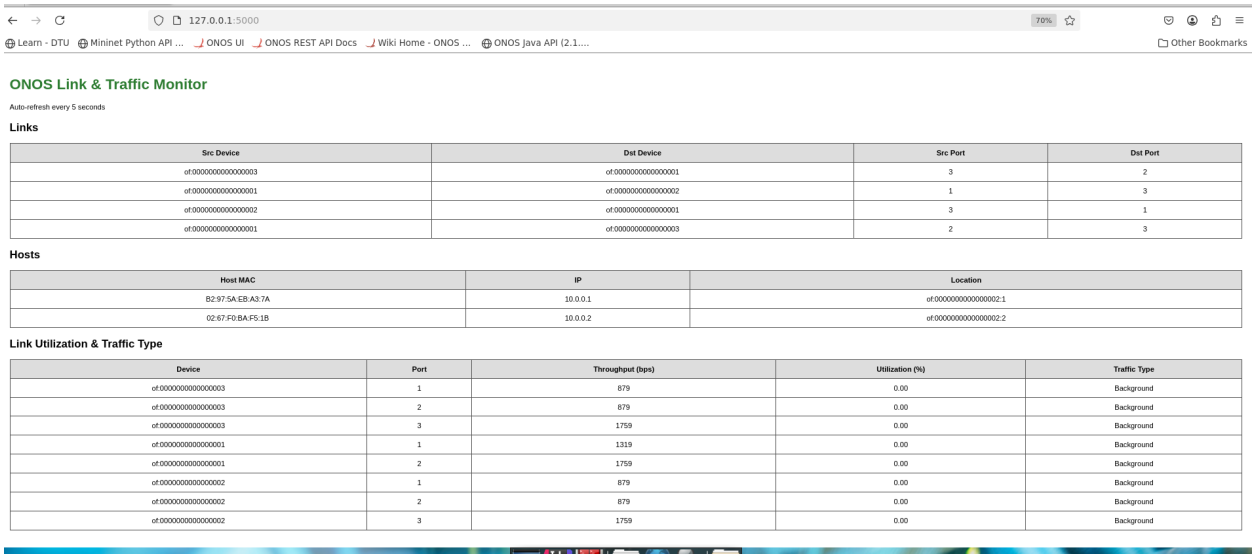

Figure 4: Host and Link details shown in Monitoring page

This confirms that external applications can effectively reuse the controllerâs global topology view without additional instrumentation. All OpenFlow switches emulated in Mininet are accurately recognized using ONOS device identifiers, and the links between switches are correctly displayed in the monitoring interface. Host information, including MAC addresses, IP addresses, and attachment points, is also correctly reported once the hosts begin generating traffic.

These results confirm that the application effectively leverages the global network view provided by ONOS. In addition, any changes to the network topology are accurately reflected after each polling interval. The observed behavior is consistent with the expected topology discovery mechanisms implemented by the ONOS controller.

5.2 Link Utilization Measurement Results

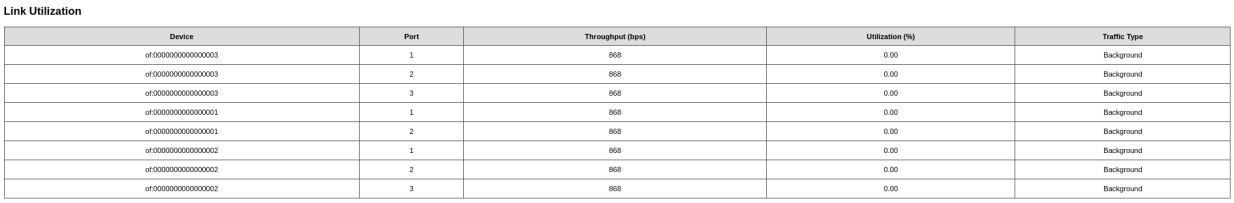

Figure 5: Link Utilization table in Idle condition

Device	Port	Throughput (bps)	Utilization (%)	Traffic Type
of:0000000000000003	1	3520038	0.40	Interactive
of:0000000000000003	2	3462779921	100.00	High Bandwidth
of:0000000000000003	3	1308	0.00	Background
of:0000000000000001	1	1308	0.00	Background
of:0000000000000001	2	1308	0.00	Background
of:0000000000000002	1	436	0.00	Background
of:0000000000000002	2	436	0.00	Background
of:0000000000000002	3	436	0.00	Background

Traffic Utilization Graph

Figure 6: Link Utilization table when active (iPerf)

The link utilization values recorded by the monitoring service accurately represent network activity during both idle and active traffic conditions. When no user traffic is present, the application still reports low but non-zero utilization on network links. This outcome is expected because of control-plane traffic, including LLDP packets for topology discovery, ARP messages, and other background processes in the Mininet environment. When traffic is generated between hosts using iPerf, there is a noticeable increase in throughput and link utilization along the forwarding path. The utilization values rise steadily during active traffic and decrease once the traffic stops. This behavior confirms that the delta-based throughput calculation is accurate and that port-level statistics effectively capture variations in network load.

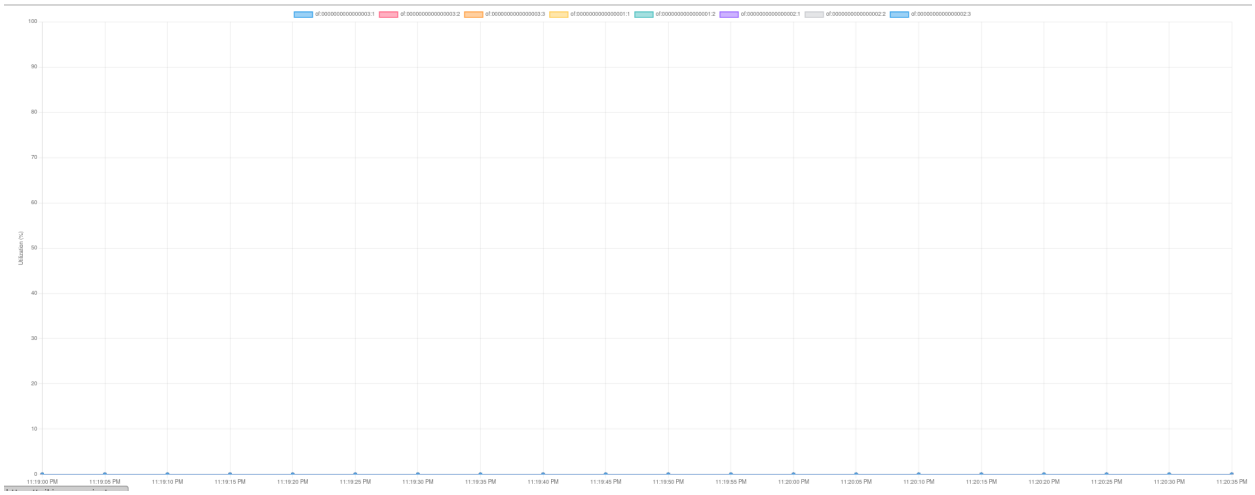

Figure 7: Traffic utilization graph in idle conditon

Figure 8: Traffic utilization graph when active (iPerf)

5.3 Traffic Classification Results

Link Utilization

Device	Port	Throughput (bps)	Utilization (%)	Traffic Type
of:0000000000000003	1	4786155	0.50	Interactive
of:0000000000000003	2	4239225913	100.00	High Bandwidth
of:0000000000000003	3	810	0.00	Background
of:0000000000000001	1	810	0.00	Background
of:0000000000000001	2	810	0.00	Background
of:0000000000000002	1	810	0.00	Background
of:0000000000000002	2	810	0.00	Background
of:0000000000000002	3	810	0.00	Background

Figure 9: Classification of traffic according to traffic load

The traffic classification mechanism is shown to operate correctly across a range of traffic loads. When no user-generated traffic is present and the network is idle, most observed traffic is classified as *background traffic*. As traffic volume increases, particularly when tools such as iPerf are used to generate load, a larger proportion of traffic is classified as *bulk data traffic* or other higher-throughput categories.

These results demonstrate that the classification process reflects real-time monitoring and assessment of dynamic network conditions. The monitoring service is therefore able to provide a semantic interpretation of raw traffic statistics. Although the classification approach is heuristic and inherently coarse-grained, it offers useful insight into the types of traffic present in the network and validates the applicability of application-level traffic analytics in Software-Defined Networking environments.

5.4 Temporal Behavior and Responsiveness

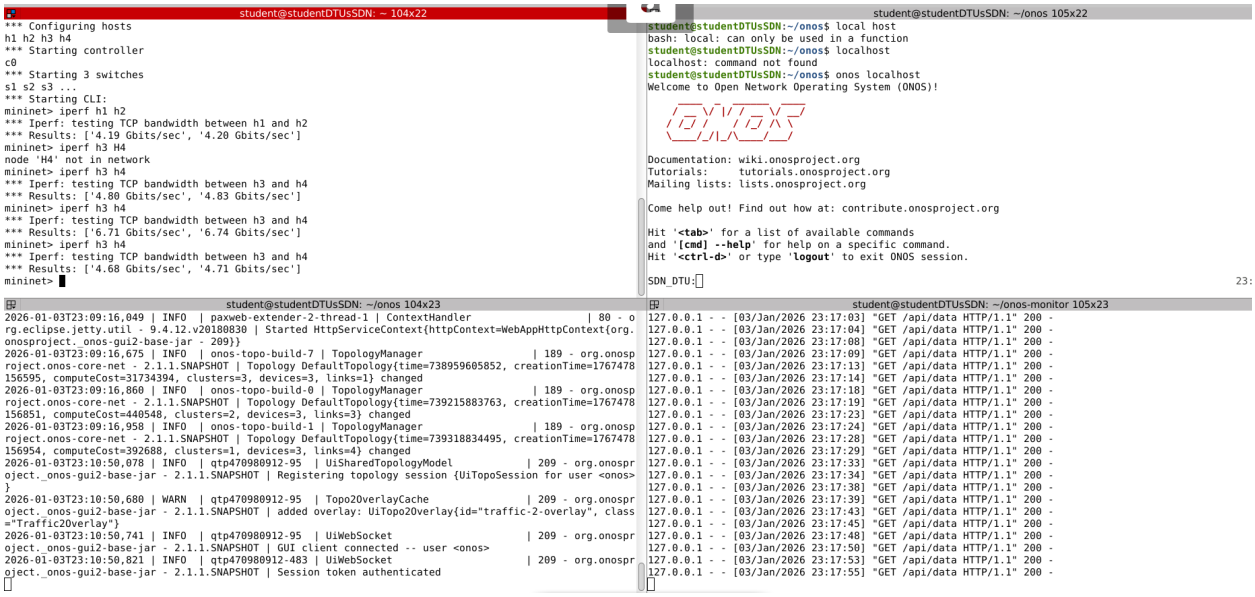

Figure 10: Monitoring App in terminal

The monitoring application provides continuous visibility into network behavior through periodic updates. Changes in traffic patterns or network topology are reflected in the user interface shortly after they occur, corresponding to the applicationâs polling intervals. This periodic polling strategy achieves a reasonable level of responsiveness without introducing unnecessary complexity, thereby maintaining stable performance during long-term operation.

Throughout all experiments, the web-based user interface remained responsive, with no observable discrepancies between backend data updates and frontend visualizations.

5.5 Overall Observations

Centralized monitoring based on an SDN controller provides accurate and consistent views of network behavior. Since the ONOS controller maintains a global view of the network and exposes a northbound interface, the proposed approach eliminates challenges associated with distributed measurements, per-device configuration, and data inconsistency.These results are consistent with prior studies on SDN-based centralized monitoring,which demonstrate improved visibility and simplified network management through controller-driven data collection [1,2].

Overall, the results confirm that the proposed solution successfully addresses the monitoring challenges identified in the problem statement.

6 Conclusion

In this project, an SDN-based network monitoring service was designed and evaluated using the ONOS controller and a Mininet emulation environment. By leveraging SDNs separation of the control and data planes, the monitoring tool was implemented as a standalone application operating above the controller. This approach enables comprehensive network visibility without requiring any modifications to the underlying hardware.

The monitoring service retrieves topology and port-level statistics directly from ONOS through REST APIs, computes link throughput, and classifies traffic patterns. Experimental evaluation shows that the system accurately captures network behavior under both idle and active traffic conditions, including the effects of control-plane overhead. A web-based interface provides a real-time view of network status, demonstrating that controller-driven monitoring is both practical and effective.

Overall, this project highlights how SDN simplifies network management through centralized control and abstraction. The modular design of the monitoring service makes it extensible, allowing future enhancements such as fine-grained flow analysis or integration with intent-based networking frameworks. The proposed solution meets all initial project objectives and provides a solid foundation for future development and research.

References

[1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[2] D. Kreutz, F. M. V. Ramos, P. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[3] Open Networking Foundation, "ONOS: A distributed SDN operating system," Technical report, Open Networking Foundation, 2014. [Online]. Available: `https://onosproject.org`